



Cégep **André-Laurendeau**

Travail pratique 2 - II

« Premier circuit avec Raspberry PI et Python »

420-555-AL

Programmation appliquée aux objets connectés

Note: / 100

À remettre le mardi 3 octobre à la fin du cours

Objectifs de l'évaluation

Dans ce travail vous allez contrôler un circuit avec votre Raspberry PI avec du code Python. Vous vous limiterez au contrôle en sortie, i.e. que vous ne ferez encore aucune lecture sur un composant. Vous aurez besoin de LEDS, de résistances, d'avertisseurs sonores et d'un interrupteur. Ce travail peut être divisé en quatre sections :

- 1- Validation de l'entrée par l'utilisateur : un nombre signé ou une phrase
- 2- Encodage d'un nombre en sa valeur binaire signée complément à deux
- 3- Encodage d'un texte en BRAILLE selon une approche **sono-lumineuse**
- 4- Codage d'une interface graphique pour contrôler votre circuit

Directives importantes

- Ce travail pratique est réalisé **individuellement**.
- Divisez clairement votre platine en régions distinctes correspondant aux sections du travail
- Modalités de remise :
 - Le 3 octobre septembre avant la fin du cours, vous devez remettre votre programme Python et votre vue schématique Fritzing sur LÉA. Vous devez aussi avoir fait corriger votre platine par l'enseignant.

Section 1 : Validation de l'entrée (20 points)

Votre programme Python ne doit accepter qu'une suite de caractères majuscules (incluant l'espace) **OU** un entier signé en format décimal. Tous les trente-sept (37) caractères acceptés sont montrés dans la liste suivante :

ABCDEFGHIJKLMNOPQRSTUVWXYZ01234567890-

à laquelle on ajoute l'espace, donc trente-huit (38) caractères en tout.

Lorsque l'utilisateur entre un nombre, ce nombre doit être compris entre -128 et +127. Voici des exemples d'entrées valides et invalides :

<u>Valides</u>	<u>Invalides</u>
CHEWING GUM	124.7
JE MANGE UNE POMME	hello
-121	128
0	MÉSANGE

Dans le cas où l'entrée est invalide, le programme affichera un message à l'écran, fera sonner un avertisseur sonore pendant une seconde et attendra une nouvelle entrée de l'utilisateur. Exemple d'interaction dans la console :

```
> Quelle est votre entrée? 124.7    Affiche un message et sonne l'avertisseur pour 1 sec.
> Quelle est votre entrée? CHEWING GUM    Exécute la section 3 sur votre platine
> Quelle est votre entrée? 11    Affiche 00001011 et exécute la section 2 sur votre platine
```

Matériel : un avertisseur sonore

Section 2 : Encodage d'un nombre décimal en binaire signée (30 points)

Dans le cas où la donnée entrée est un nombre signé, vous devez d'abord calculer la valeur binaire du nombre décimal reçu en entrée. Cette valeur binaire sera ensuite affichée à l'écran sur huit LEDS alignées côte-à-côte, ces LEDS représentant l'octet signé en complément à deux. Un « 1 » sera représenté par une LED **allumée**, un « 0 » par une LED **éteinte**. Par exemple, le nombre décimal 11 (onze) sera affiché comme 00001011 sur votre écran et comme ceci sur votre platine :



Le nombre -127 sera affiché comme 10000001 sur votre écran et comme ceci sur votre platine :



Ces affichages seront d'une durée de cinq secondes avant de s'éteindre.

Les trois bits de poids faibles (ceux à droite) seront aussi affichés en utilisant une seule LED pleine couleur. Les couleurs utilisées pour ces trois bits sont les suivantes :

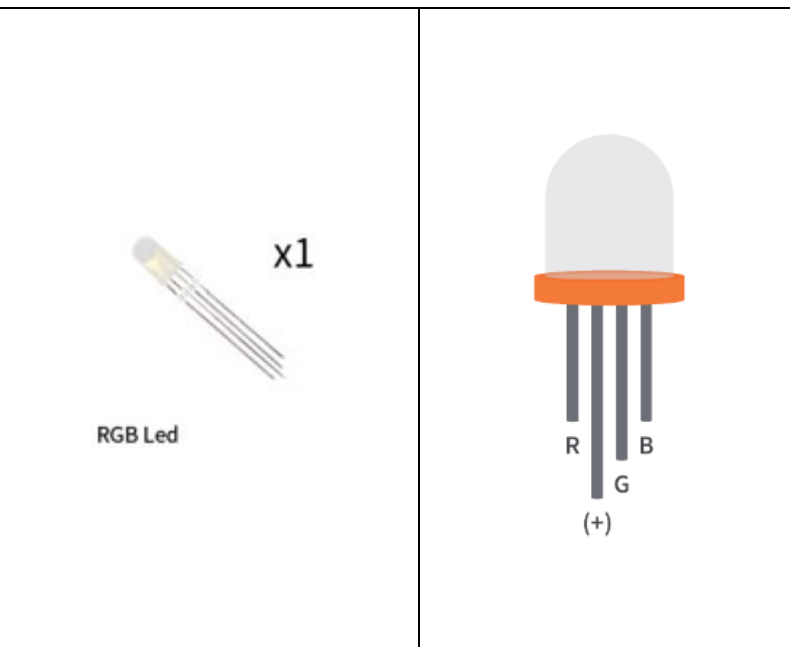
3 derniers bits	couleur
000	éteinte
001	bleu
010	vert
011	cyan
100	rouge
101	magenta
110	jaune
111	blanc

Matériel : huit LEDs et une RGBLED.

Une RGBLED :

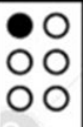
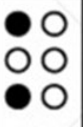
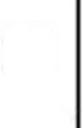
- Alimentation de 3.3 volts
- C'est un composant à **anode commune** :
 - Les trois broches pour les couleurs ont chacun leur branchement GPIO (n'oubliez pas les résistances)
 - L'anode est branchée sur le 3.3 volts
- Configuration :
 - Le paramètre `active_high` doit être à false

Connexion test [ici](#) : attention cette connexion test utilise une RGBLED à cathode commune.



Section 3 : Encodage d'une phrase en Braille (30 points)

Dans le cas où la donnée entrée est un mot ou une phrase, vous devez l'encoder en Braille. L'encodage que nous allons utiliser permet de communiquer une phrase sous la forme de signaux lumineux¹ et/ou de sons. Dans ce code, les lettres sont encodées à l'aide de points dans une grille de trois lignes par deux colonnes :

 A	 B	 C	 D	 E	 F	 G	 H	 I	 J
 K	 L	 M	 N	 O	 P	 Q	 R	 S	 T
 U	 V	 W	 X	 Y	 Z	1	2	3	4
5	6	7	8	9	0			 —	 space

Sur votre circuit, chaque symbole est représenté par une grille de trois par deux LEDs (couleur à votre choix). La lettre « C » est donc encodée avec les deux LEDs sur la première ligne allumée et les autres éteintes. De plus, chaque symbole est encodé avec une trame sonore relié à cette grille : en parcourant la grille de gauche à droite et de haut en bas, un point noir fera bipper l'avertisseur sonore (intervalle de 0.1 seconde) durant une demie seconde alors qu'un point blanc l'activera pendant une demie seconde. Pour séparer les lettres, les six LEDs et l'avertisseur seront désactivés (éteints) durant une seconde. Pour séparer les mots, autrement dit lorsqu'on rencontre un espace, les six LEDs et l'avertisseur seront désactivés (éteints) durant deux secondes.

¹¹ Inutile pour une personne avec handicap visuel, mais nous ferons l'hypothèse que chaque signal lumineux s'accompagne d'une protubérance de la LED, palpable au touché.

La phrase « LE CHAT » est donc encodée comme suit:

Symbole	LEDs	Avertisseur	Secondes
L		Bip 0.5 sec Actif 0.5 sec Bip 0.5 sec Actif 0.5 sec Bip 0.5 sec Actif 0.5 sec	0-3 secondes
Inter-lettre		Éteint	3-4 secondes
E		Bip 0.5 sec Actif 0.5 sec Actif 0.5 sec Bip 0.5 sec Actif 0.5 sec Actif 0.5 sec	4-7 secondes
Espace (inter-mot)		Éteint	7-9 secondes
C		Bip 0.5 sec Bip 0.5 sec Actif 0.5 sec Actif 0.5 sec Actif 0.5 sec Actif 0.5 sec	9-12 secondes
Inter-lettre		Éteint	12-13 secondes
H		Bip 0.5 sec Actif 0.5 sec Bip 0.5 sec Bip 0.5 sec Actif 0.5 sec Actif 0.5 sec	13-16 secondes
Inter-lettre		Éteint	16-17 secondes
A		Bip 0.5 sec Actif 0.5 sec Actif 0.5 sec Actif 0.5 sec Actif 0.5 sec Actif 0.5 sec	17-20 secondes
Inter-lettre		Éteint	20-21 secondes
T		Actif 0.5 sec Bip 0.5 sec Bip 0.5 sec Bip 0.5 sec Bip 0.5 sec Actif 0.5 sec	21-24 secondes

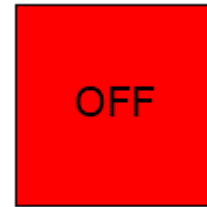
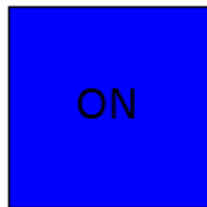
Finalement, vous ajouterez un interrupteur pour activer/désactiver l'avertisseur sonore. Cet interrupteur ne sera pas contrôlé par votre programme Python, il le sera seulement manuellement.

Un interrupteur : La seule fonction de cet interrupteur pour ce travail est de désactiver/débrancher l'avertisseur sonore		
--	---	---

Matériel : six LEDs, un avertisseur sonore et un interrupteur.

Section 4 : Interface graphique (20 points)

Fournissez une interface graphique de base (Turtle ou autre) avec deux boutons :



En pressant sur ON : toutes les LEDS et avertisseurs seront à ON

En pressant sur OFF : toutes les LEDS et avertisseurs seront à OFF

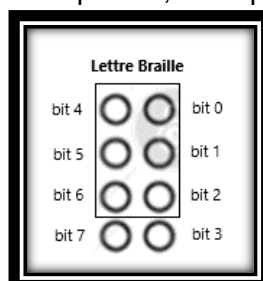
Vous aurez besoin de threads pour exécuter les deux tâches suivantes simultanément :

1. Votre boucle d'interaction avec l'utilisateur dans la console
2. La gestionnaire d'événement pour les clics sur les boutons

Vous pouvez vous inspirer du code Python à la page suivante pour la manipulation des threads. Ce code est fourni avec l'énoncé.

Partage des LEDs

Pour éviter la multiplicité de LEDs sur votre platine, vous pouvez les partager de la façon suivante :



```

1 # Turtle: Gestion d'événements (clics) avec threading
2 from turtle import *
3 import keyboard, threading
4 from time import sleep
5 tortue = Turtle()
6 led = True
7 # Fonction qui va s'occuper des clics
8 def attrapeClic(x, y): # x et y sont les coordonnées du clic
9     global led
10    if x >= -50 and x <= 50 and y >= -50 and y <= 50:
11        led = not led
12 # On déclare quelle méthode va attraper les clics gauches
13 onscreenclick(attrapeClic,1) # 1: clic gauche
14 # on dessine un rectangle rouge en plein centre
15 col = 'red'
16 tortue.speed(0)
17 tortue.goto(-50, -50)
18 tortue.fillcolor(col)
19 tortue.begin_fill()
20 for _ in range(4):
21     tortue.forward(100)
22     tortue.left(90)
23 tortue.end_fill()
24 tortue.hideturtle()
25 # Définir une fonction qui tournera dans son propre thread
26 def autre_tache():
27     while True:
28         print("La led est " + str(led))
29         sleep(1)
30         # Vous pouvez mettre votre code ici
31 # Créer un thread pour une autre tâche
32 autre_tache_thread = threading.Thread(target = autre_tache)
33 # Démarrer l'autre thread
34 autre_tache_thread.start()
35 mainloop() # pour garder le programme actif, sinon il s'arrête

```

BARÈME	Fonctionnement	Circuit Fritzing	Total
Section 1 : validation	/12	/8	/100
Section 2 : encodage d'un nombre	/18	/12	
Section 3 : encodage d'une phrase	/18	/12	
Section 3 : interface graphique	/20		
Commentaires			